Universidad San Carlos de Guatemala

Centro Universitario de Occidente

División de Ciencias de la Ingeniería

Lenguajes Formales y de Programación

Ing. Daniel Gonzalez



Manual Técnico
PromptZal

Guillermo Emanuel Montejo Martínez
Carné: 202430624

Quetzaltenango, 28 de octubre de 2026

Introducción

El presente documento describe los aspectos técnicos del compilador desarrollado para el lenguaje PromptZal, como parte del Proyecto 2 del Laboratorio de Lenguajes Formales y de Programación.

PromptZal es un lenguaje que permite crear prompts estructurados mediante archivos con extensión .pz, en los que se definen agentes, variables e instrucciones de inteligencia artificial. El compilador transforma estos programas en un texto final llamado *promptzal*, listo para utilizarse en una herramienta de inteligencia artificial.

Este proyecto amplía el analizador léxico desarrollado en el Proyecto 1, incorporando las demás fases del compilador. Para ello, se utiliza JFlex en el análisis léxico y un analizador sintáctico descendente recursivo que verifica la estructura del programa y permite detectar múltiples errores sintácticos. Durante este proceso, se construye un árbol de sintaxis abstracta (AST), que sirve para generar la tabla de símbolos y validar aspectos semánticos, como la declaración de identificadores y las referencias a agentes.

Finalmente, el sistema cuenta con una interfaz gráfica desarrollada en Java Swing que permite editar y compilar programas, visualizar los resultados y consultar los errores encontrados. También genera reportes HTML sobre los tokens, errores, tabla de símbolos, AST y estadísticas, además de representaciones gráficas del autómata finito determinista y del árbol de sintaxis abstracta mediante Graphviz.

Requisitos del Sistema

Requisitos mínimos para ejecutar el proyecto:

- Sistema Operativo: Windows, Linux o macOS
- Java Development Kit (JDK): Versión 17 o superior
- Memoria RAM: 512 MB mínimo
- Espacio en disco: 50 MB mínimo
- Un archivo de texto con extensión .pz para compilar
- Graphviz (opcional): necesario únicamente para generar las imágenes del AFD y del árbol de sintaxis abstracta (AST). Debe estar instalado y disponible en el PATH del sistema. Sin Graphviz, el compilador funciona con normalidad y los reportes HTML se generan igual; el reporte del AST muestra solo su versión en texto.

Para compilar el proyecto:

- Apache Maven 3.6 o superior
- NetBeans IDE 17 o superior (recomendado)
- Conexión a internet en la primera compilación: Maven descarga automáticamente el plugin de JFlex (jflex-maven-plugin), por lo que no es necesario instalar JFlex por separado. El analizador léxico se genera a partir del archivo src/main/jflex/promptzal.flex en cada compilación.

Para ejecutar el .jar

java -jar PromptZalV3-1.0-SNAPSHOT.jar

Compilar desde NetBeans:

1. Abrir el proyecto en NetBeans
2. Click derecho en el proyecto
3. Seleccionar “Clean and Build”
4. El JAR se genera en la carpeta target

Compilar desde Consola

1. Abrir CMD o PowerShell
2. Navegar a la carpeta raíz del proyecto: cd ruta\del\proyecto
3. Ejecutar: mvn clean package
4. El JAR se genera en la carpeta target

Para ejecutar desde consola :

- Navegar a la carpeta target: cd ruta\del\proyecto\target
- Ejecutar: java -jar PromptzalV3-1.0-SNAPSHOT.jar

1) Análisis Léxico

El analizador léxico fue generado utilizando la herramienta JFlex. Su función es leer el flujo de caracteres de entrada, descartar espacios y comentarios, y agrupar los caracteres válidos en tokens.

Archivo Jflex

El archivo Lexer.jflex está dividido en tres secciones:

- **Configuración y Código de Usuario:** Se define el paquete, las importaciones, y un método token() que compensa el inicio en cero de JFlex sumando 1 a yyline y yycolumn para reportar posiciones exactas.
- **Macros y Expresiones Regulares:** Se definen patrones base como ESPACIO, COMENTARIO_LINEA, LETRA, DIGITO, ID, CADENA y NUMERO.
- **Reglas Léxicas:** Se mapean los lexemas a sus respectivos tipos (TipoToken). Se utiliza la regla comodín [^] al final para capturar cualquier carácter no reconocido, registrarlo en una lista de errores léxicos y continuar el análisis sin detener la ejecución.

Tabla de Terminales

Terminal (Tipo Token)	Descripción / Lexema
DIRECTIVA	@modelo, @rol, @formato
AGENTE	Palabra reservada AGENTE
CONTEXTO	Palabra reservada contexto
VARIABLE	Palabra reservada variable
EJECUTAR	Palabra reservada EJECUTAR
EXPORTAR	Palabra reservada EXPORTAR
COMANDO_IA	PREGUNTAR, GENERAR, RESUMIR, ANALIZAR, TRADUCIR, CLASIFICAR, EXTRAER
CARGAR	Función CARGAR
CONECTOR	SOBRE, DESDE, EN, COMO

Terminal (Tipo Token)	Descripción / Lexema
FLECHA	Operador ->
IGUAL	Operador =
MAS	Concatenación +
LLAVE_A / LLAVE_C	Delimitadores { y }
PAR_A / PAR_C	Delimitadores (y)
COMA	Separador ,
ID	Identificadores (nombres de variables, agentes, o palabras de unidades)
CADENA	Texto entre comillas dobles "..."
NUMERO	Enteros y decimales
EOF	Fin de archivo

2) Análisis Sintáctico

La fase sintáctica se implementó mediante un Parser Descendente Recursivo programado completamente a mano, sin uso de generadores. Cada no terminal de la gramática corresponde a un método en la clase Parser.java.

<programa> -> <directivas> <agentes> <ejecuciones> <exportar> EOF

<directivas> -> DIRECTIVA CADENA <directivas> | ε

<agentes> -> <agente> <mas_agentes>

<mas_agentes> -> <agente> <mas_agentes> | ε

<agente> -> AGENTE ID LLAVE_A <contexto> <variables> <comandos> LLAVE_C

<contexto> -> CONTEXTO IGUAL CADENA

<variables> -> VARIABLE ID IGUAL <expresion> <variables> | ε

<comandos> -> <comando> <mas_comandos>

<mas_comandos> -> <comando> <mas_comandos> | ε

<comando> -> COMANDO_IA <expresion> <conectores> FLECHA ID

<conectores> -> CONECTOR <expresion> <conectores> | ε

<expresion> -> <termino> <mas_terminos>

<mas_terminos> -> MAS <termino> <mas_terminos> | ε

<termino> -> CADENA | ID | NUMERO <unidad> | <cargar>

<unidad> -> ID | ε <cargar> -> CARGAR PAR_A CADENA PAR_C

<ejecuciones> -> EJECUTAR ID <mas_ejecuciones>

<mas_ejecuciones> -> EJECUTAR ID <mas_ejecuciones> | ε

<exportar> -> EXPORTAR ID <mas_ids>

<mas_ids> -> COMA ID <mas_ids> | ε

Manejo de errores(Modo Pánico)

Para evitar que el compilador se detenga en el primer error sintáctico, se implementó la recuperación en "Modo Pánico". Cuando el método match() detecta un token inesperado:

1. Registra el error (token encontrado vs. esperado).
2. Lanza una excepción personalizada PanicException que interrumpe la evaluación de la regla actual.
3. El bloque padre atrapa la excepción y llama a recuperarModoPanico().
4. Este método descarta tokens consecutivamente hasta encontrar un token de sincronización (AGENTE, LLAVE_C, COMANDO_IA, VARIABLE, EJECUTAR o EXPORTAR) para retomar el análisis.

3) Diseño del Arbol de Sintaxis Abstracta (AST)

Mientras el parser valida la gramática, instancia clases de Java para representar el programa en memoria. El modelo del AST consta de los siguientes nodos:

- Programa: Nodo raíz. Contiene listas de directivas (String), agentes (Agente), ejecuciones (String) y exportaciones (String).
- Agente: Representa un bloque AGENTE. Almacena su nombre, contexto, y listas de sub-nodos variables y comandos.
- Variable: Almacena nombre, valor y tipoOrigen (CADENA, NUMERO, CARGAR, etc.).
- Comando: Define una instrucción de IA. Guarda la operacion (ej. PREGUNTAR), el parametroPrincipal, una lista de conectores y la variableDestino (->).
- Conector: Nodo hoja que asocia un tipo (SOBRE, EN, etc.) con su valor.

4) Análisis Semántico y Tabla de Símbolos

El AST es recorrido por la clase AnalizadorSemantico para validar la lógica y llenar la Tabla de Símbolos.

Tabla de Símbolos

Es una lista de objetos Simbolo que almacenan:

- **Nombre:** Identificador único.
- **Tipo:** AGENTE, VARIABLE_DECLARADA, o RESULTADO_COMANDO.
- **Ámbito:** "Global" (para agentes) o el nombre del agente al que pertenece una variable.
- **Fila:** Línea de declaración para referencias de error.

Reglas Semánticas implementadas

Unicidad de Agentes: Se valida que no existan dos agentes con el mismo nombre en el ámbito Global.

- **Unicidad de Variables:** Se valida que no existan dos variables declaradas con el mismo nombre dentro del mismo agente.

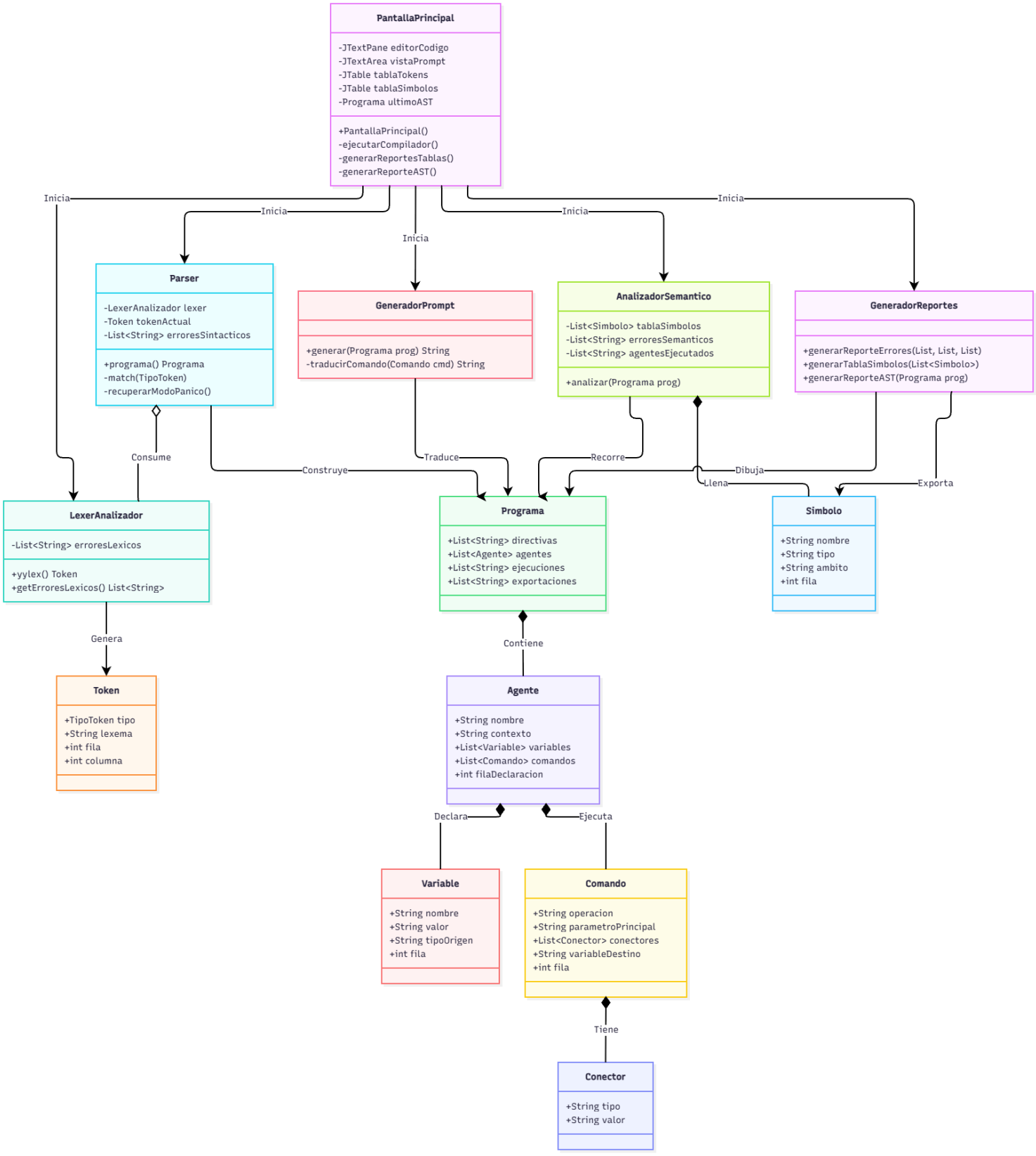
- **Uso de Identificadores (Declaración Previa):** Si un argumento principal o un valor en un conector es un identificador puro (no es número, ni cadena con comillas, ni concatenación), se busca en la tabla de símbolos del agente correspondiente. Si no existe, se reporta error. Se exime de esta regla el primer parámetro del comando GENERAR.
- **Validación de Ejecuciones y Exportaciones:** EJECUTAR valida que el agente llamado exista. EXPORTAR valida que la variable solicitada exista en los ámbitos de los agentes previamente ejecutados.

5)Generación del PromptZal

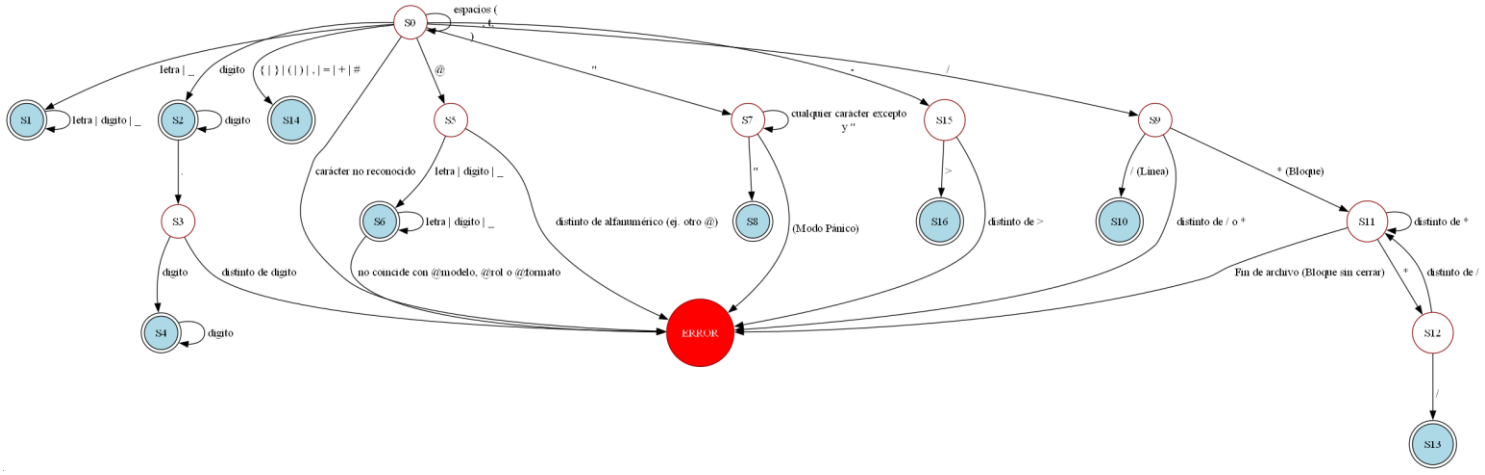
Si el análisis semántico es exitoso, la clase GeneradorPrompt recorre el AST validado y traduce las estructuras a lenguaje natural.

1. Directivas: Se imprimen como metadatos en el encabezado del reporte Markdown.
2. Agentes: Por cada agente invocado en EJECUTAR, se redacta su Contexto, se listan sus Variables disponibles (indicando si requieren adjuntar archivos en caso de CARGAR) y se enumeran sus Comandos.
3. Traducción de Comandos: Se utiliza un bloque switch sobre el tipo de operación (PREGUNTAR, RESUMIR, etc.) para inyectar el parámetro principal y los conectores dentro de plantillas predefinidas en lenguaje natural. Por ejemplo: Basandote en [SOBRE], responde: [parametro].

6) Diagrama De Clases


PantallaPrincipal
-JTextPane editorCodigo
-JTextArea vistaPrompt
-JTable tablaTokens
-JTable tablaSimbolos
-Programa ultimoAST
+PantallaPrincipal()
-ejecutarCompilador()
-generarReportesTablas()
-generarReporteAST()
Inicia
Inicia
Inicia
Inicia
Inicia
Parser
-LexerAnalizador lexer
-Token tokenActual
-List<String> erroresSintacticos
+programa() Programa
-match(TipoToken)
-recuperarModoPanico()
GeneradorPrompt
+generar(Programa prog) String
-traducirComando(Comando cmd) String
AnalizadorSemantico
-List<Simbolo> tablaSimbolos
-List<String> erroresSemanticos
-List<String> agentesEjecutados
+analizar(Programa prog)
GeneradorReportes
+generarReporteErrores(List, List, List)
+generarTablaSimbolos(List<Simbolo>)
+generarReporteAST(Programa prog)
Consume
Construye
Traduce
Recorre
Dibuja
Llena
Exporta
LexerAnalizador
-List<String> erroresLexicos
+yylex() Token
+getErroresLexicos() List<String>
Programa
+List<String> directivas
+List<Agente> agentes
+List<String> ejecuciones
+List<String> exportaciones
Simbolo
+String nombre
+String tipo
+String ambito
+int fila
Genera
Contiene
Token
+TipoToken tipo
+String lexema
+int fila
+int columna
Agente
+String nombre
+String contexto
+List<Variable> variables
+List<Comando> comandos
+int filaDeclaracion
Declara
Ejecuta
Variable
+String nombre
+String valor
+String tipoOrigen
+int fila
Comando
+String operacion
+String parametroPrincipal
+List<Conector> conectores
+String variableDestino
+int fila
Tiene
Conector
+String tipo
+String valor

7) AFD



8) Sistema de reportes

El proyecto incluye un motor independiente (GeneradorReportes.java) para la creación de la documentación en formato web (HTML5) estipulada en los requerimientos.

Reportes estaticos

Al solicitar los reportes, el sistema recopila las listas de errores (ultimosErrLex, ultimosErrSin, ultimosErrSem) y la Tabla de Símbolos (ultimaTablaSimbolos). Utiliza StringBuilder para concatenar etiquetas HTML y CSS embebido (estilo moderno con flexbox y badges de colores), y los escribe directamente en disco en la carpeta

- **Traducción a DOT:** El método generarReporteAST recorre el árbol en memoria (AST) y asigna identificadores únicos (ej. n0, n1) a cada nodo. Las relaciones padre-hijo (ej. Programa -> Agente) se traducen a la sintaxis .dot de Graphviz (n0 -> n1;).
- **Llamada al Sistema:** Utilizando la clase ProcessBuilder, el IDE ejecuta el comando de terminal dot -Tpng -o AST.png AST.dot.
- **Sincronización:** Mediante el método waitFor(), el hilo de Java espera a que el sistema operativo termine de dibujar la imagen PNG antes de generar el archivo ReporteAST.html que la incrusta.

9) Herramientas y Entorno de Construcción

- **Lenguaje Base:** Java (JDK 25 configurado).
- **Gestor de Dependencias y Construcción:** Apache Maven. Se configuró el maven-assembly-plugin en el archivo pom.xml para empaquetar el proyecto en un único archivo ejecutable (.jar) que incluye todas sus dependencias.
- **Generador Léxico:** JFlex (integrado en el flujo para construir el archivo LexerAnalizador.java).
- **Requisito del Entorno de Ejecución:** Para la correcta visualización del AST, el sistema huésped debe tener instalado el paquete *Graphviz* y contar con el comando dot configurado en sus Variables de Entorno del Sistema (PATH).